WISH FINDER

기억 속 장면을 찾아 주는
근거 기반 AI Agent

광주 3반 G078 문희주

AI 서비스 Mini Project · 과제 2 (신규 Agent) · 첫 적용 대상: NCT WISH 영상 3편

장면을 설명하면, 원본 자막으로 검증한 시점을 찾아 준다

구분	내용
문제	제목도 대사도 모르고 "그 장면"만 기억난다
해결	검색 → 원본 cue 대조 → 의미 판정 → 영상 시점과 앞뒤 자막
제약	자막은 영어 번역본이고 화자 표시가 없어, 화자는 단정하지 않는다



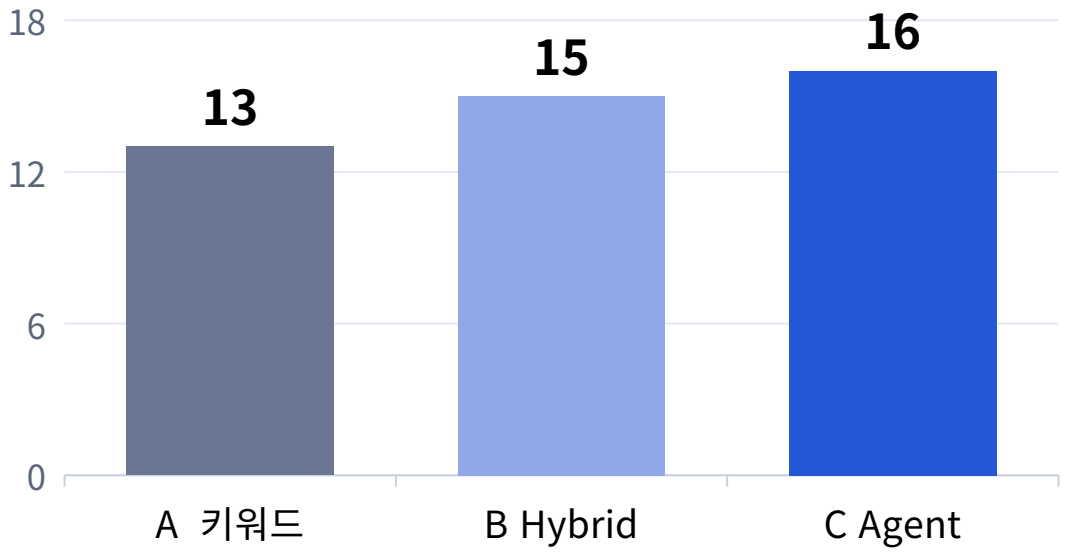
결과 화면: 검증된 장면 카드, 영상, 원본 앞뒤 자막

세부 task

파싱·청킹	Hybrid 검색	LangGraph	MCP 서버	API·Vue·SSE	평가·추적
cue 3,750 → 청크 869	BM25 + bge-m3, RRF	6개 노드, 재검색 ≤ 2	읽기 전용 도구 4개	진행 단계 실시간	A/B/C 20문항

결과: 16/18을 맞혔고, 근거 없는 화자 단정은 0건

A / B / C 1위 적중 수 (18문항 중)



A 키워드 · B Hybrid(키워드+의미) · C Agent(Hybrid+검증·재검색)
같은 영어 검색어 조건 · 1~3문항 차이, 1회 실행이라 우위 단정 안 함

개선 전/후 (OpenAI, 같은 평가셋)

지표	v2	v3	변화
1위 적중 (18문항)	16	16	같음
화자 질문 처리 (4문항)	2	3	+1
LLM 호출 (20문항)	50	42	−16%
평균 지연	2.88초	2.57초	−11%
입력 토큰 (C+C-ko)	56,941	70,770	+24%
틀린 장면을 "찾음"	0건	1건	+1

v3 개선: 화자 질문 판정 기준 · 번역 자막 대응 · 중복 재검색 제거 · 검색어 재작성

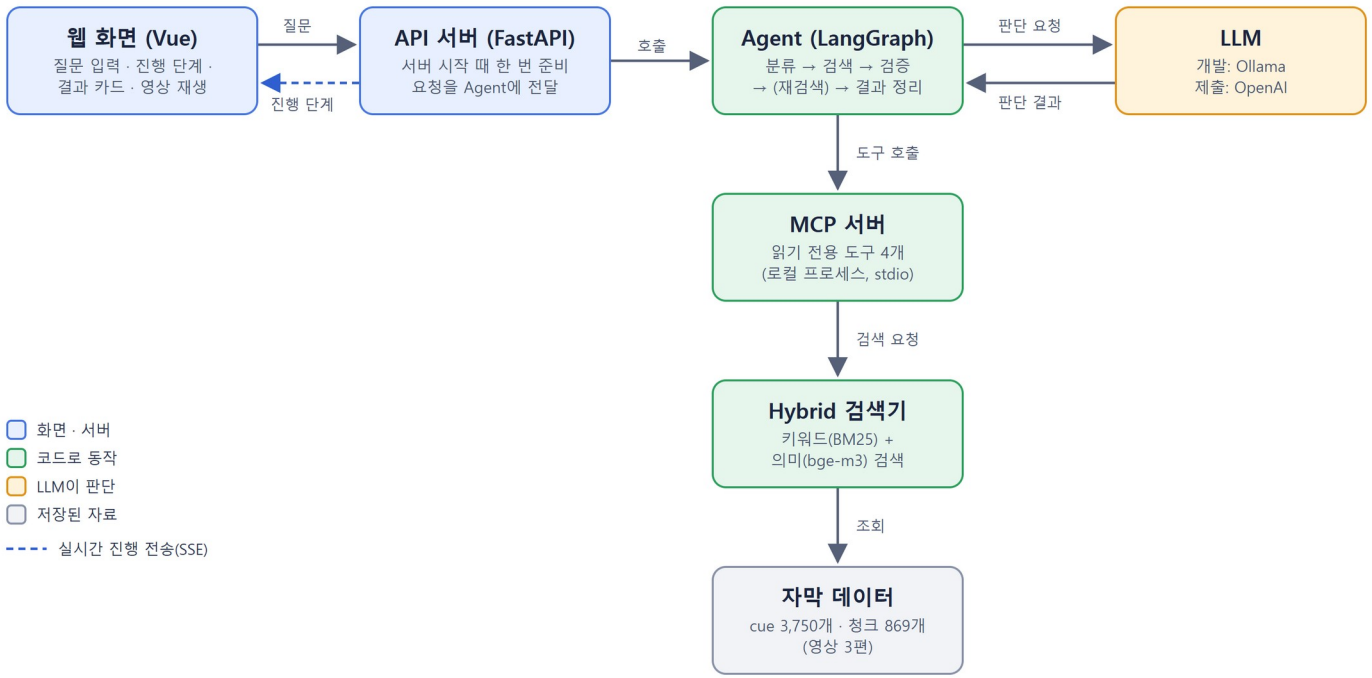
0건
근거 없이 화자를 단정한 경우

16 → 16
이중 첫 검색을 켜도 결과 같음.
검색만 80 → 98회
기본 꺼짐으로 설정

20초/10초
청크 길이 비교 후 유지
1위 적중: 10/5=11 · 20/10=11
30/15=9 · 40/20=5

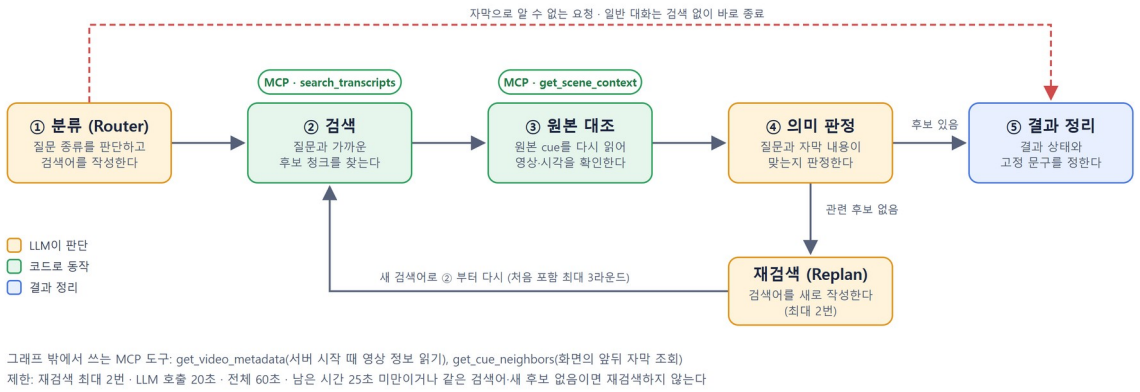
한계
후속 질문·번역 어휘 차이
Multi-Agent 미구현

LangGraph · MCP · RAG를 하나의 흐름으로 엮었다



구성	내용
LangGraph	분류 · 검색 · 검증 · 재검색 흐름
MCP	공식 SDK 2.x, 로컬 stdio, 읽기 전용 도구 4개
RAG	BM25 + bge-m3 + RRF, 원본 cue 근거
의존성 주입	생성자 주입, LLM을 환경변수로 교체
LangSmith	노드·LLM 호출·토큰 추적
모듈화	agent · mcp · retrieval · services 분리
테스트	백엔드 178개 · 프런트 43개

Agent Graph: 경로는 코드가, 판단은 LLM이



그래프 밖에서 쓰는 MCP 도구: get_video_metadata(서버 시작 때 영상 정보 읽기), get_cue_neighbors(화면의 앞뒤 자막 조회)
제한: 재검색 최대 2번 · LLM 호출 20초 · 전체 60초 · 남은 시간 25초 미만이거나 같은 검색어·새 후보 없음이면 재검색하지 않는다

설계 원칙	내용
다음 노드	조건 함수가 정한다 (add_conditional_edges)
LLM 판단	분류 · 판정 · 검색어 재작성, 세 곳
인용·시각	코드가 원본 cue에서 생성, LLM은 쓰지 않음
구조	단일 Agent 워크플로 (Multi-Agent 미구현)
상한	재검색 2번 · LLM 20초 · 전체 60초

```
graph = StateGraph(FinderState)
for name in ['router', 'search', 'context',
             'verification', 'replan', 'finalize']:
    graph.add_node(name, self._node(name))

graph.add_conditional_edges('router',
    lambda s: 'finalize' if s['intent'] in
              {'unsupported', 'chat'} else 'search')
graph.add_conditional_edges('verification',
                            self._after_verification)
graph.add_conditional_edges('replan',
    lambda s: 'finalize' if s.get('stop_replan')
              else 'search')

def _after_verification(self, state):
    return 'replan' if state.get('can_replan') \
           else 'finalize'

MAX_REPLAN_ROUNDS = 2
```

backend/app/agent/graph.py (발췌, 줄 바꿈만 조정)

MCP: 자막은 도구로만 접근하고, 검증은 원본 cue로 다시 한다

도구	쓰는 곳
search_transcripts	Agent · 검색 단계
get_scene_context	Agent · 원본 대조
get_video_metadata	API 서버 · 시작 시
get_cue_neighbors	화면 · 앞뒤 자막

원칙	내용
공식 MCP SDK 2.x	로컬 프로세스, stdio 통신
읽기 전용	파일 경로를 인자로 받지 않는다
검색 결과를 믿지 않는다	인용·시각은 원본 cue로 다시 확인

```
@server.tool(structured_output=True)
def search_transcripts(
        query: str, mode='hybrid',
        top_k=3, video_ids=None
        ) -> SearchReply:
    """Search registered transcripts.
    Returns cue-linked candidates,
    not answers."""
    with search_gate.acquire():
        _, retriever = resources()
        return retriever.search(
            query, mode, top_k, video_ids)

# Agent verifies each candidate by
# re-reading the source cues:
@server.tool(structured_output=True)
def get_scene_context(video_id,
        start_seconds, window_seconds=20)
        -> ContextReply:
```

backend/app/mcp/server.py (발췌, 타입 표기 단순화)

RAG: 두 검색을 RRF로 합치고 원본 cue로 되돌린다

자막 cue 3,750줄

▼

청크 869개 (20초 단위, 10초 겹침)

▼

BM25 상위 20 + bge-m3 상위 20

▼

RRF로 순위 합치기

▼

원본 cue로 되돌리기

```
fused = {}
for rank in ranks:    # BM25 list, dense list
    for position, i in enumerate(rank, 1):
        fused[i] = fused.get(i, 0) \
                   + 1 / (60 + position)

for i in sorted(fused,
                key=lambda i: (-fused[i], i)):
    chunk = self.chunks[i]
    ...  # near-duplicate chunks removed
    candidate = Candidate(chunk=chunk,
                          score=fused[i])
```

backend/app/retrieval/hybrid.py (발췌, 줄 바꿈만 조정)

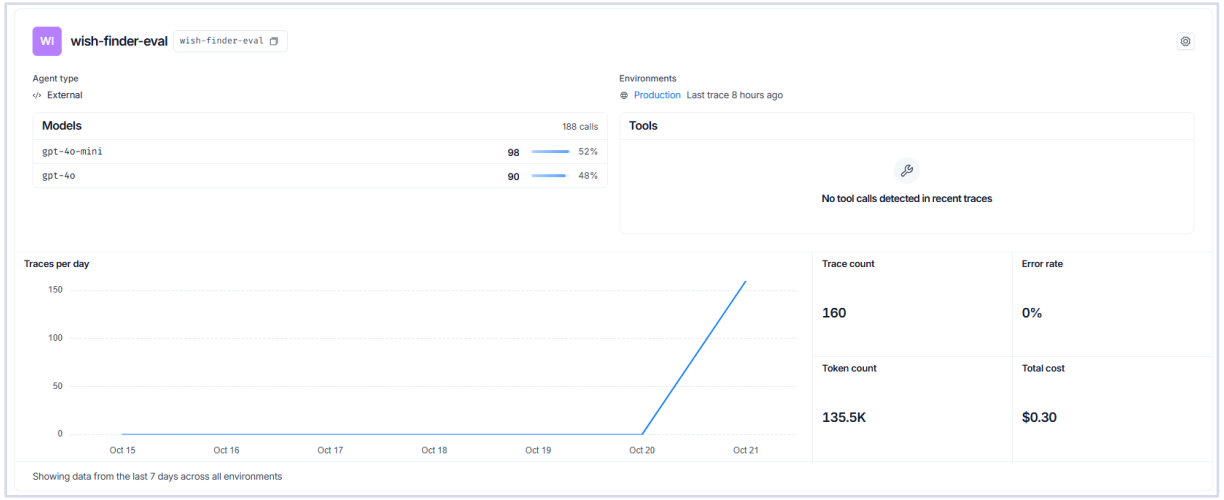
점수 = Σ 1 / (60 + 순위)

청크 길이 비교(B, 18문항): 10/5=11 · 20/10=11 · 30/15=9 · 40/20=5
→ 20/10 유지. 60은 미비교

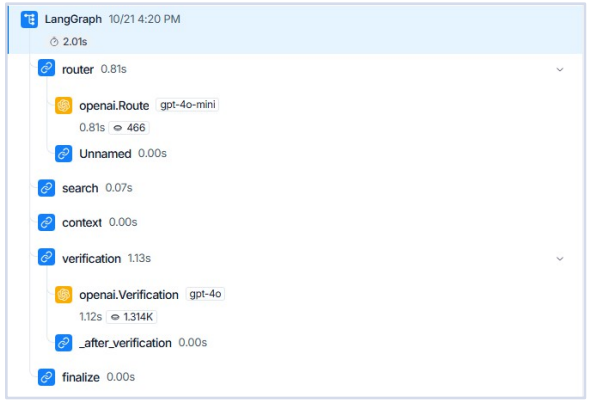
[백업] 한계와 트레이드오프

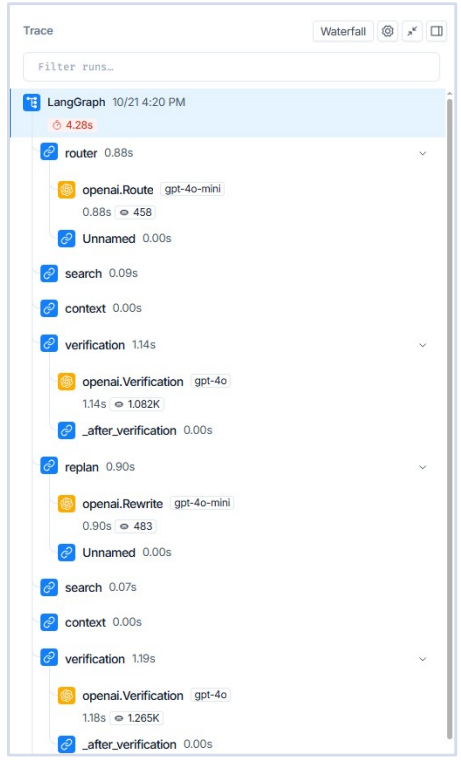
구분	한계	근거
측정	18문항·질문당 1회, Router 검색어가 실행마다 달라져 1~2문항 차이는 노이즈일 수 있다	보고서 3.4·4.3절
측정	정답 라벨은 어시스턴트가 원본 cue를 검토해 제안하고 사용자가 승인. 영상 대조는 9곳 표본	보고서 4.3절
성능	"찾음" 응답 17건 중 1건이 틀린 장면(16/17). 원본 대조는 의도 일치를 보장하지 않는다	보고서 3.2·4.2절
성능	정답 청크가 후보 3개 밖(10위·18위)인 2문항은 판정 이전 단계의 한계. 번역 어휘·후속 질문도 약점	보고서 4.2절
비용	입력 토큰 +24%, 평균 지연 2.6초(Hybrid 0.2초대), 이중 첫 검색은 검색 +1회	보고서 3.2·3.3절
범위	Multi-Agent 미구현·미비교, 영어 번역 자막, 화자 단정 불가	보고서 4.1·4.3절

[백업] LangSmith로 노드별 시간·LLM 호출·토큰을 추적했다



평가 2회 합계: 트레이스 160 · 모델 호출 188 · 토큰 135.5K · 비용 $0.30 (LangSmith 계산값)





왼쪽: 재검색 없는 건 2.01초 · LLM 2회
오른쪽: 재검색 1번 건 4.28초 · LLM 4회
MCP 도구 호출은 LangSmith에 따로 잡히지 않는다 (한계)

[백업] 의존성 주입과 테스트

```
class FinderAgent:
    def __init__(self, session, metadata, model, *,
                 mode='hybrid',
                 tool_call=call_tool,
                 timeout=60,
                 telemetry=None,
                 dual_query=False, ...):
        self.session = session    # MCP session
        self.model = model        # Ollama | OpenAI

# production: real session + build_model()
# tests: FakeSession + ScriptedModel
```

backend/app/agent/graph.py (발췌)

LLM 교체: WISH_LLM_BACKEND 환경변수 하나

Ollama(개발) ↔ OpenAI(제출), 나머지 코드는 그대로

백엔드 테스트 영역	개수
Agent 그래프	55
LLM 교체	19
자막·검색	37
설정·API·실시간 전송	33
평가 도구	32
실제 MCP 통신	2

백엔드 178개 통과 · 프런트 43개 통과

모델은 가짜로 대신해 과금 없음, 실제 stdio MCP 서버 테스트 포함

[백업] 이중 첫 검색: 번역 어휘 차이를 질문 원문 검색으로 보완

문제

Router 검색어: "manitto"
자막: "Secret Santa"
→ 같은 의미인데 못 찾음

방법

Router 검색어 + 질문 원문
두 검색을 함께 실행하고
순위를 번갈아 채워 후보 3개

확인

마니또: 못 찾음 → 찾음
대가: 질문당 약 1~2초 증가
평가셋 20문항: 16 → 16, 이득 없음

구현 중에 발견한 성능 저하

코드 리뷰 반영 중 점수 합산 방식으로 바꿨더니 단위 테스트는 통과했지만 실제 자막에서 정답이 후보 3개 밖으로 밀렸다 ("구운 오징어": 45:53 → 42:41). 원인을 확인하고 순위를 번갈아 채우는 방식으로 고쳐 테스트로 고정했다.

[백업] 기술 요소 정리와 Multi-Agent를 쓰지 않은 이유

기술 요소	이 프로젝트에서
Multi-Agent	미구현 (단일 Agent 워크플로)
MCP	공식 SDK 2.x, 도구 4개, stdio
RAG	BM25 + bge-m3 + RRF, 원본 cue
의존성 주입	생성자 주입, LLM 환경변수 교체
LangSmith	노드·LLM 호출·토큰 추적
모듈화	agent / mcp / retrieval / services
단위 테스트	백엔드 178 · 프런트 43

Multi-Agent를 쓰지 않은 이유

일이 순서대로 짧고 병렬로 할 일이 없다

Supervisor는 LLM 호출·지연·실패 지점을 늘린다

실제 원인은 구조가 아니라 어휘였다

설계안 (구현·측정하지 않음)

Supervisor가 Agent를 고르고, Search Agent가 도구를 스스로 고르며, Verifier Agent는 판정만 한다